

ParkPulse – Technology Stack Document

Project Name: ParkPulse – Smart Parking, Smarter Cities

Version: 1.0

Project Type: Full-Stack Web Application + IoT Integration

1. Overview

ParkPulse is a cloud-based smart parking management system that enables users to locate, reserve, and manage parking spaces in real time. The application combines a modern React frontend with Supabase services and IoT-enabled parking sensors to deliver a seamless parking experience.

The system is designed to be scalable, secure, and deployable for real-world smart parking environments such as malls, hospitals, airports, office campuses, and smart cities.

2. Technology Stack Overview

Layer	Technology	Purpose
Frontend	React.js	Build responsive user interface
Styling	Tailwind CSS	Modern and responsive UI design
Language	JavaScript (ES6+)	Application logic and functionality
Backend Services	Supabase	Authentication, Database & Realtime
Database	PostgreSQL (Supabase)	Store application data
Authentication	Supabase Auth	Secure user login and registration
Realtime Engine	Supabase Realtime	Live parking slot updates
Maps	Google Maps API	Location and navigation
QR Code	QRCode Library	Generate booking QR codes
Version Control	Git & GitHub	Source code management

Deployment

Vercel

Hosting and Continuous Deployment

IDE

Visual Studio Code

Development environment

3. Frontend Technology

React.js

Why React?

React is a modern JavaScript library for building interactive web applications.

Purpose

- User Authentication
- Dashboard
- Parking Search
- Live Parking Map
- Reservation System
- Payment Interface
- Admin Dashboard

Advantages

- Component-based architecture
 - Fast rendering using Virtual DOM
 - Easy state management
 - Scalable project structure
 - Excellent community support
-

4. Styling Framework

Tailwind CSS

Purpose

Tailwind CSS is used to create a clean, responsive, and modern user interface.

Features

- Mobile-first design
 - Fast UI development
 - Responsive layouts
 - Utility-first CSS
 - Easy customization
-

5. Programming Language

JavaScript (ES6+)

JavaScript powers the complete frontend logic of ParkPulse.

Responsibilities

- User interactions
 - Form validation
 - Reservation logic
 - Slot allocation
 - Payment calculations
 - API communication
 - QR generation
 - Notifications
-

6. Backend Platform

Supabase

Supabase provides all backend services without requiring a custom server.

Services Used

- PostgreSQL Database
- Authentication
- Realtime Updates
- Storage
- Row Level Security (RLS)
- REST API
- SQL Editor

Why Supabase?

- Open-source Backend-as-a-Service
 - PostgreSQL database
 - Built-in authentication
 - Real-time synchronization
 - Secure API generation
 - Easy integration with React
-

7. Database

PostgreSQL (Supabase)

A relational database is used to store all application data.

Tables

Users

Stores user information.

Fields

- User ID
 - Name
 - Email
 - Phone Number
 - Password (Managed by Supabase Auth)
-

Parking Lots

Stores parking locations.

Fields

- Parking ID
- Parking Name
- Address
- Latitude
- Longitude
- Total Slots

Parking Slots

Stores slot information.

Fields

- Slot ID
- Parking ID
- Slot Number
- Floor
- Status
- Sensor ID

Reservations

Stores booking information.

Fields

- Reservation ID
- User ID
- Slot ID
- Booking Time
- Expiry Time
- QR Code

Payments

Stores payment history.

Fields

- Payment ID
- Reservation ID
- Amount
- Payment Status
- Payment Method

Parking History

Stores previous parking sessions.

Fields

- History ID
 - User ID
 - Entry Time
 - Exit Time
 - Duration
 - Total Cost
-

8. Authentication

Supabase Authentication

Features

- Email Login
- Google Login
- Password Reset
- Session Management
- Secure Authentication Tokens

Benefits

- High security
 - Easy implementation
 - User session handling
 - Role-based access
-

9. Real-Time Updates

Supabase Realtime

Whenever a parking slot changes:

Available

↓

Occupied



Available

Every connected user immediately receives the update.

Benefits

- Live parking availability
 - No page refresh required
 - Better user experience
-

10. Google Maps API

Purpose

Provide navigation and nearby parking locations.

Features

- Current location
 - Route guidance
 - Distance calculation
 - Travel time
 - Parking location display
-

11. QR Code System

Each successful reservation generates a unique QR Code.

The QR Code contains:

- Reservation ID
- User ID
- Slot Number
- Booking Time

Used For

- Entry verification
- Exit verification

- Secure parking access
-

12. Version Control

Git

Git tracks every change made during development.

Benefits

- Code history
 - Rollback support
 - Branch management
 - Team collaboration
-

GitHub

GitHub stores the source code online.

Features

- Repository hosting
 - Collaboration
 - Pull Requests
 - Issue Tracking
 - Backup
 - Version Management
-

13. Deployment

Vercel

The ParkPulse application is hosted using Vercel.

Deployment Workflow

Developer writes code

↓

Git Commit

↓

Push to GitHub

↓

Vercel detects changes

↓

Automatic Build

↓

Automatic Deployment

↓

Live Website Updated

Benefits

- Automatic deployment
- HTTPS enabled
- Fast global CDN
- Continuous Integration
- Easy rollback

14. Development Tools

Tool	Purpose
Visual Studio Code	Code Editor
Git	Version Control
GitHub	Code Repository
Vercel	Deployment
Chrome DevTools	Debugging

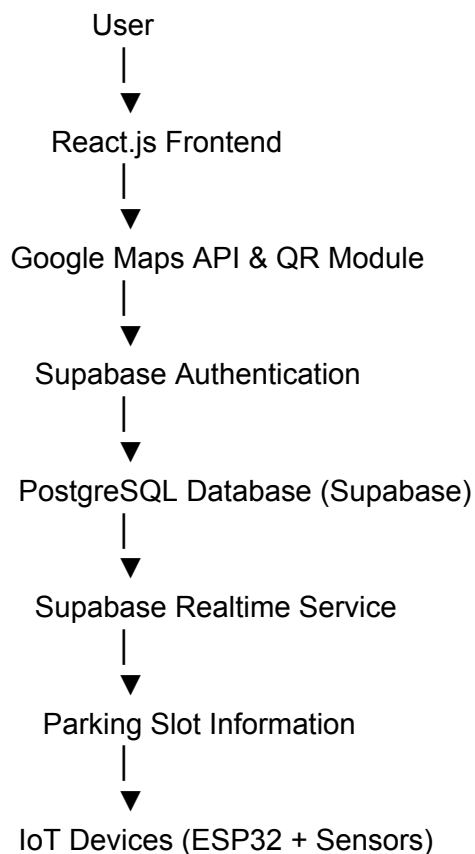
Postman

API Testing

Figma

UI/UX Design

15. Overall System Architecture



16. Why This Tech Stack?

React.js

- Fast and interactive UI
- Component-based development
- Excellent scalability

Supabase

- Built-in authentication
- PostgreSQL database

- Real-time synchronization
- Secure APIs

PostgreSQL

- Reliable relational database
- Supports complex queries
- High performance

Google Maps API

- Accurate navigation
- Nearby parking discovery

GitHub

- Secure source code management
- Collaboration and version control

Vercel

- One-click deployment
 - Automatic updates from GitHub
 - Fast and reliable hosting
-

17. Future Technology Enhancements

- Progressive Web App (PWA)
 - Push Notifications
 - Automatic Number Plate Recognition (ANPR)
 - Machine Learning for parking demand prediction
 - EV charging station integration
 - Voice assistant support
 - Smart city integration
 - Mobile application using React Native
 - Analytics dashboard with charts and reports
-

Conclusion

The selected technology stack provides a modern, scalable, and production-ready foundation for ParkPulse. React.js delivers a responsive user experience, Supabase

simplifies backend development with authentication, PostgreSQL, and real-time capabilities, while GitHub and Vercel enable professional version control and continuous deployment. Together, these technologies create a secure, efficient, and maintainable smart parking solution suitable for both a capstone project and future real-world deployment.